

AI Safety Critical Runtime Monitoring

Edoardo Sarri

Resiliency Real-Time and Certification

Giugno 2026



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Introduction

Integrating AI models into systems that requiring predictable and safe behavior:

- **Challenge:** Black-box nature of AI models.
- **Issue:** Missing of structured standards and methodologies.
- **Starting point:** The **Artificial Intelligence for Safety-Critical Systems in Industrial and Transportation Domains: A Survey** paper.
- **Repository:** **GitHub**.

Introduction

Terminology

- **AI and Machine Learning:** Algorithmic paradigm that allows models to learn patterns from data.
- **AI Safety:** Methodologies to deploy AI in safety-critical systems in compliance with safety standards, avoiding/mitigating catastrophic harm.
- **Functional Safety (FuSa):** *Part of the overall safety of a system that assures the freedom from unacceptable risk (ISO 26262).* Explicitly, is the characteristics of the system that bring back and maintain the system in a safe state.

Overview

Types of AI (TAI)

- **Connectionists:** Optimization and gradient-based (e.g., NNs and transformers).
- **Bayesian:** Probabilistic outcomes.
- **Symbolists:** Logic-based and explainable (e.g., decision trees and prolog).
- **Analogizers:** Similarity-based (e.g., k-NN and SVM).

Overview

AI Automation Levels

The EASA define three level of automation:

- **Level 1:** Collaborative robots, that work side-by-side with humans without independence. For example a robot harm that help a worker.
- **Level 2:** Autonomous systems that operate in a closed (i.e., predefined, where requirements are well known) environments. For example a drone that map a closed environment like a factory.
- **Level 3:** Autonomous systems that operate in a open environments and that can adapt their goal and behaviour. For example a autonomous driving car.

Overview

Safety Standards

The SIS (Safety Integration System) have the objective to mitigation risks through programmable functions embedded in the system.

- **Measures:** SIL (IEC 61508), ASIL (ISO 26262), DAL (DO-178C).
- **Gap:** Critical levels require 10^{-9} /h failure probability; the AI model reaches around 99% accuracy. In a AI-based System (e.g., ADAS) we can have a system failure without a software or hardware failure

Overview

When Introducing AI

The ISO 5469 defined three scenario that correlate the type of AI and their usage in a complex system.

- **Class I:** Deterministic AI (e.g., symbolist) $\Rightarrow$ Formally verifiable $\Rightarrow$ Complies with traditional FuSa.
- **Class II:** Partial verifiable AI model $\Rightarrow$ Requires to be monitored by a runtime checker (*Safety Bag* techniques) that take a safe decision in case of AI error.
- **Class III:** Totally unverifiable AI model $\Rightarrow$ Safety responsibility cannot be taken by the engineers (e.g., complex ADAS).

Overview

How Introducing AI

- **Product:** AI-based safety functions that are trained offline and then embedded in the system.
- **Runtime:** AI models that update/adapt online during system execution.
- **Process:** AI models are used to design/develop/test the system.

Overview

Product

AI-based safety functions that are trained offline and then embedded in the system.

We speak about *intended functionality*: a high level of instructions set rather than specific statements.

- Explainable and Interpretable: *Class I* scenario.
- Back box: *Class II* scenario.

Overview

Runtime

AI models that update/adapt online during system execution.

- **Safety Bag:** A RunTime checker chooses a safety behaviour rather than the AI choice.
- **Safe Adaptation:** Both model and training algorithm are conformed to a standard.
- **Limited adaptation:** The parameters chainging is clapped.
- **Limited Actuation:** The magnitude of the AI action is clapped.
- **Library-based Offline:** A deterministic choice (i.e, if/else) choose among a set of model that are conform to safety standard.

Overview

Process

AI models are used to design/develop/test the system.

- **Specification:** NLP (e.g., transformers) models for hazard analysis.
- **Design/Testing:** Code and test code base.
- **AutoML:** Reinforcement learning for hyperparameter tuning.

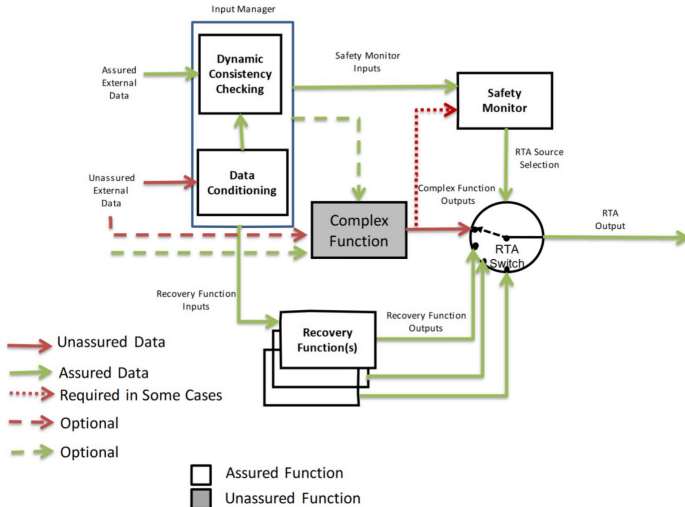
Runtime Monitoring

Runtime Assurance (RTA)

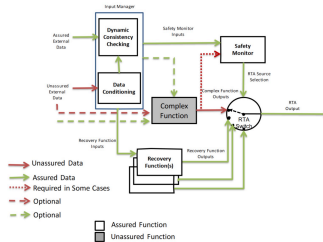
Runtime Assurance (RTA) system is able to handle the complexity of AI model without handle it directly.

- **Concept:** Instead of trying to certify a highly complex or non-deterministic AI module, we can wrap it in a runtime monitor (i.e., a runtime checker) that switches to an assured recovery function if it recognises a failure.
- **Modularity:** The system remains certified even if we totally change the AI model: the certification is not based on the quality of AI.
- **Versatility:** Can be applied at any system level (e.g., controllers, actuators) and with one monitor for each SIS.

Runtime Monitoring - Architectures



Runtime Monitoring - Architectures



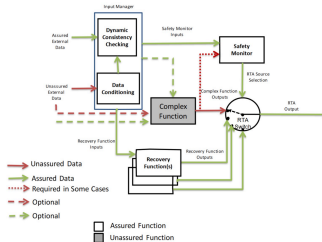
Input Manager

Validates unassured data. These data can derive both from AI model and from the extern.

Complex Function

Untrusted AI module that perform the SIS.

Runtime Monitoring - Architectures



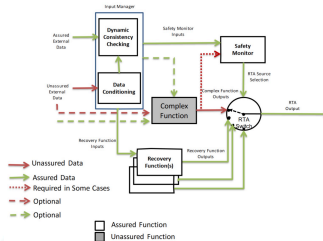
Recovery Function

Assured module that perform the SIS.

Safety Monitor

Assured module that detects AI failures.

Runtime Monitoring - Architectures

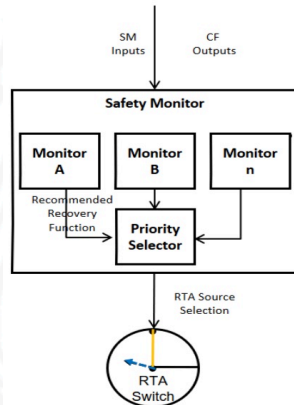
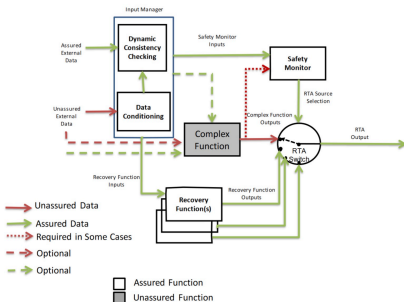


Switch

Assured routing from AI to recovery function.

Runtime - KooN

In the case of very complex function, one monitor cannot cover all possible failure and is mandatory the usage of multiple monitor.



Implementation

The Advanced Driver Assistance System (ADAS) is based on:

- **Ego Car (E):** Controlled by AI (i.e., RL PPO algorithm) or Safety Monitor.
- **Leader Car (L):** Independent vehicle ahead with unknown acceleration $a_L(t)$.
- **Safety Bag Pattern:** AI operates until $d(t) \leq d_{critical}$, then Recovery Function brakes at maximum deceleration.

Implementation

Stochastic Leader Acceleration

$$a_L(t) = \begin{cases} \text{clip}(A \sin(\omega t + \phi), -8.0, 3.0) & \text{if } t < t_{brake} \\ a_{max_brake} & \text{if } t \geq t_{brake} \text{ and } v_L(t) > 0 \\ 0 & \text{otherwise} \end{cases}$$

- $t_{brake} = \tau \sim \mathcal{U}(5, 20)$ with $p = 0.3$, else ∞ .
- $A \sim \mathcal{U}(1.0, 5.0) \text{ m/s}^2$, $\omega \sim \mathcal{U}(0.1, 0.4) \text{ rad/s}$, $\phi \sim \mathcal{U}(0, 2\pi)$.

Kinematics Rules ($\Delta t = 0.1 \text{ s}$, discrete)

- $v(t+1) = \max(0, v(t) + a(t) \cdot \Delta t)$

Implementation

State Space: $S_t \in \mathbb{R}^3$

- **Distance:** $d(t) = x_L(t) - x_E(t)$
- **Relative Velocity:** $v_{rel}(t) = v_L(t) - v_E(t)$
- **Ego Velocity** $v_E(t)$

Initial Conditions ($t = 0$)

- $x_E(0) = 0 \text{ m}$, $d(0) \sim \mathcal{U}(5.0, 20.0) \text{ m}$
- $v_L(0), v_E(0) \sim \mathcal{U}(10.0, 20.0) \text{ m/s}$, $a_E(-1) = 0.0 \text{ m/s}^2$

Implementation

Safety Monitor

Computes minimum distance required to avoid collision: check the $d(t) \leq d_{critical}$ condition.

$$d_{critical} = \max \left(0, v_E(t) \cdot \Delta t + \frac{v_E(t)^2 - v_L(t)^2}{2 \cdot |a_{max_brake}|} \right)$$

- $v_E(t) \cdot \Delta t$ accounts for the system's 1-tick reaction delay.

Recovery Function

- **Fail-safe behavior:** Apply maximum deceleration ($a_E = -8.0 \text{ m/s}^2$).

Implementation

Reward

The reward Function is defined as

$R_t = R_{distance} + R_{comfort} + R_{penalty}$ and takes into account:

- $R_{distance} = -\alpha |d(t) - d_{target}|$: Define the target distance from Ego to Leader (i.e., $d_{target} 10m$).
- $R_{comfort} = -\beta |a_E(t-1) - a_E(t)|$: Penalize sudden braking.
- $R_{penalty} = -20$ if $d(t) \leq d_{critical}$: Penalize entering critical zone.
- $R_{terminal} = -1000$ if $(d(t) \leq 0)$ or $(d(t) > 50 m)$: Penalize simulation failure.

Implementation

Results

We have three different scenarios:

- **Success:** The Ego follow the leader within the correct interval.
- **Collision:** Lower bound critical distance reached.
- **Leader Lost:** Upeer bound critical distance reached.

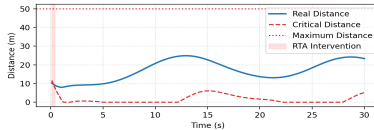
Implementation - Results

AI Safety Critical Runtime Monitoring

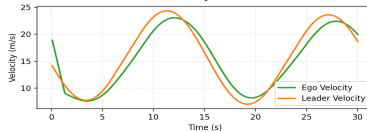
Simulation completed successfully!

Metric	Value
Total Time	30.00 s (300 ticks)
AI Decisions	297 (99.0%)
RTA Interventions	3 (1.0%)
Minimum Distance	8.01 m
Average Distance	17.03 m (Target: 10m)
Maximum Distance	24.90 m
Maximum Ego Velocity	23.04 m/s (83.0 km/h)
Average Ego Velocity	15.23 m/s (54.8 km/h)

Distance Chart



Velocity Chart



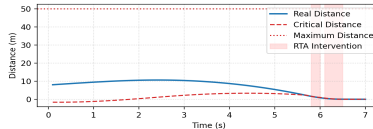
Implementation - Results

AI Safety Critical Runtime Monitoring

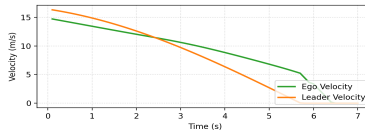
Collision detected!

Metric	Value
Total Time	7.00 s (70 ticks)
AI Decisions	64 (91.4%)
RTA Interventions	6 (8.6%)
Minimum Distance	0.00 m
Average Distance	7.00 m (Target: 10m)
Maximum Distance	10.63 m
Maximum Ego Velocity	14.70 m/s (52.9 km/h)
Average Ego Velocity	8.75 m/s (31.5 km/h)

Distance Chart



Velocity Chart



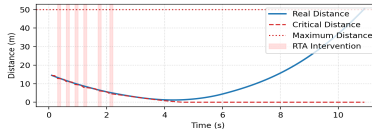
Implementation - Results

AI Safety Critical Runtime Monitoring

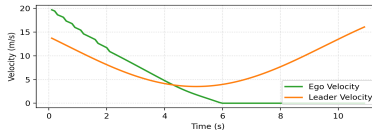
Leader vehicle lost (tracking failed)!

Metric	Value
Total Time	10.90 s (109 ticks)
AI Decisions	103 (94.5%)
RTA Interventions	6 (5.5%)
Minimum Distance	1.20 m
Average Distance	12.91 m (Target: 10m)
Maximum Distance	50.40 m
Maximum Ego Velocity	19.69 m/s (70.9 km/h)
Average Ego Velocity	4.67 m/s (16.8 km/h)

Distance Chart



Velocity Chart



AI Safety Critical Runtime Monitoring

Edoardo Sarri

Thanks



UNIVERSITÀ
DEGLI STUDI
FIRENZE